

Name: Kaushal Kumar Ray

PRN: 240205241023

Branch: B.Tech CSE AI & ML

Semester: IV

Subject: Operating System

① What is an operating system? Explain its role as an interface between user and hardware.

→ An operating system (OS) is system software that manages computer hardware resources and provides service to application program and users.

Role as Interface:

The OS acts as a mediator between the user and the hardware.

Flow:-

User → Application Program → operating system → Hardware.

The OS control CPU, memory, storage, and I/O devices, making hardware usable without the user needing to understanding low-levels details.

② Explain different types of O.S with suitable examples:-

a) Batch operating system

- In this system, similar jobs are grouped and executed in batches without user interaction. It reduces idle time but does not provide immediate response.

ex: Early IBM mainframe systems.

b) Time-sharing O.S

- Allows multiple users to access the system simultaneously. CPU time is divided into small time slices, provides quick response time.

ex: UNIX, Linux.

c) Distributed operating system

- manages a group of independent computers and makes them appear as a single system.

Resources are shared across machines.

ex: Amoeba.

d) Real-time O.S

- provides guaranteed response within a fixed time limit.
- used in critical systems like robotics and medical devices.

ex: VxWorks, RTLinux.

e) Multiprocessing O.S

- uses two or more processors for execution
- Improves performance and reliability.

ex: window, Linux (multi-core systems).

f) Multitasking O.S

- Allows execution of multiple tasks at the same time.
- Improves productivity & responsiveness.

ex: window, MacOS.

g) Multiprogramming O.S

- keeps multiple programs in memory and switches CPU among them to increase utilization

ex: Early UNIX systems.

③ Describe the main functions of an O.S.

- i) Process management: create, scheduling, termination.
- ii) memory management: Allocation & deallocation
- iii) File management: organize & manage files & directories.
- iv) Device management: controlling I/O devices.
- v) Security & Protection: preventing unauthorized access.
- vi) CPU scheduling: Deciding which process gets CPU time.

④ Explain operating system services provided to users and programs.

→ Program execution

- I/O operations
- File system management
- Communication (Inter-Process communication)
- Error detection and handling
- Resource allocation
- Protection & security mechanisms

⑤ Discuss different OS structures:

⇒ a) Layered structure

- OS is divided into layers where each layer performs a specific function.
- Lower layers handle hardware; upper layers interact with users.

b) Monolithic structure

- Entire OS runs as a single large program in kernel mode.
- All services are directly accessible.
- Fast but less modular.

c) Microkernel Architecture

- only essential function runs in kernel mode (CPU scheduling, memory management), other services runs in user space.
- More secure and modular.

⑥ Compare Monolithic and Microkernel operating systems.

Feature	Monolithic	Microkernel
Kernel size	Large	Small
Performance	High	Slightly lower
Security	less	more
Modularity	Low	High
Maintenance	Difficult	Easier

⑦ What are system calls? Explain different types with examples.

→ System calls provide an interface between user program and the OS.

Types:

- i) Process control - `fork()`, `exit()`
- ii) File management - `open()`, `read()`, `write()`
- iii) Device management - `ioctl()`
- iv) Information maintenance - `getpid()`
- v) communication - `pipe()`, `socket()`

⑧ Define a process: Differentiate between a Program & a Process

→ A Process is a Program in execution.

Program

- Passive entity
- Stored on disk
- Static

Process

- Active entity
- Loaded in memory
- Dynamic

⑨ Explain the Process concept with suitable diagram.

→ When a Program is loaded into memory and executed, it becomes a process.

Process Structure:

- Text (code) section
- Data Section
- Stack
- Heap
- Program Counter

Simple representation:

Program → loaded into memory → Executing Process

Q10) Describe different states of process:

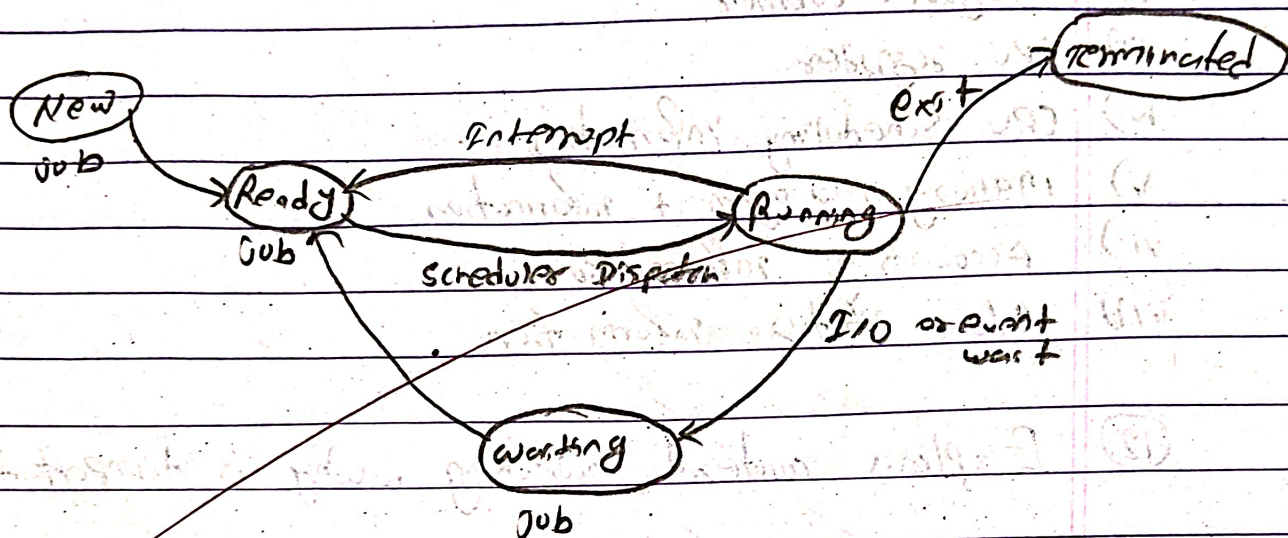


Fig: Life-cycle of a process

- New - process is being created
- Ready - waiting for CPU
- Running - currently executing
- waiting - waiting for I/O
- Terminated - Execution finished

Q11) What is PCB? process control block? explain its components

→ Each process is represented in the OS by a process control block. "PCB" also called a task control block. It contains many pieces of information associated with a specific process including these:-

- i) Process state
- ii) Program counter
- iii) CPU register
- iv) CPU scheduling information
- v) memory management information
- vi) Accounting information
- vii) I/O status information.

(12) Explain context switching - why is it important?

→ context switching is the process of saving the state of the current process and loading the state of another process.

Importance:

- Enables multitasking
- Ensures fair CPU sharing
- Improves system responsiveness

(13) Different between process scheduling and process switching.

- | Process scheduling | Process switching |
|------------------------|---|
| - Selects next process | - Changes execution from one process to another |
| - Decision process | - Technical state-saving operation |

Page _____

(14) Define thread. How it is different from a process?

→ A thread is the smallest unit of execution within a process.

Process	Thread
- Separate memory	- Shared memory
- Slower creation	- Faster creation

(15) Explain various states of thread.

→ The various states of thread are perfectly matching with the process.

- New: The task is just created.
- Ready: Job is ready to be executed.
- Running: The job is running & doing the task.
- Blocked / waiting: The job needs something to complete or is waiting for other task to complete.
- Terminated: The small task is ~~over~~ completed.

(16) Advantages of using threads.

→ Faster execution

- Better system responsiveness
- Efficient communication
- Resource sharing
- Improved CPU utilization.

(17) Types of threads:

a) user-level threads

- managed by user-level libraries.
- fast to create but kernel is unaware.

b) kernel-level threads

- managed by operating system.
- better scheduling support but higher overhead.

(18) Compare processes and threads.

Feature	Process	Thread
Memory	Separate	Shared
Communication	IPC required	Direct
creation time	High	low
overhead	High	low
Fault isolation	Strong	weak

(B)